

MESS

Product & Engineering Blueprint

PRD • Technical Requirements • App & Website Flows • Database Schema • API Contract • Implementation Plan • Security & QA

Brand tagline: “Your Meals. Your Choice.”

Prepared as an implementation handoff for Antigravity
Version 1.0 | 24 October 2026

How to use this blueprint

This document combines the product and engineering requirements. The accompanying ZIP contains the same sections as editable Markdown, the proposed SQL migration, diagrams and Antigravity execution prompts.

Open the MESS project root in Antigravity. Give it the contents of prompts/ANTIGRAVITY_MASTER_PROMPT.md and keep this package in the repository so the agent can read the requirements directly.

Important: the SQL is a proposed initial schema, not a production migration to run blindly. Validate it in a development/staging project and test explicit grants, RLS, payments and state transitions before production.

Contents

1. Product Requirements Document
2. Technical Requirements Document
3. App and Website Flows
4. Backend Schema
5. API Contract
6. Implementation Plan
7. Security, Testing and Release

Scope summary: 3 separate Flutter Android apps (Customer, Provider, Delivery Partner) + Next.js website with customer/provider/admin route groups + Supabase/PostgreSQL backend and secure server-side functions.

01 — Product Requirements Document (PRD)

Product: MESS

Tagline: Your Meals. Your Choice.

Document status: Build brief v1.0

Initial market: College students, hostel/PG residents, young professionals and local tiffin customers in India.

1. Product summary

MESS is a local food subscription marketplace connecting customers with hotels, messes, restaurants and individual tiffin providers. Customers can buy a fixed-period meal plan, a meal-credit plan or a one-time meal. They choose pickup or paid delivery. Each eligible meal is recorded using a secure QR/OTP redemption flow, so the customer, provider and platform have a consistent record of meals used, balance, delivery and payment.

MESS is not merely a QR scanner. The core product is subscription management + verified meal redemption + optional delivery + provider operations.

2. Problem statement

Customers often pay for meals through informal monthly arrangements with limited visibility into plan expiry, meals used, holidays, delivery charges and complaints. Providers often track subscriptions and meal consumption manually. Customers and providers need a lightweight system that creates clear records without adding excessive work to the meal handover.

3. Product goals

1. Let a customer discover and compare nearby participating providers and plans.
2. Make plan conditions, dates, meal limits, cancellation terms and total charges visible before purchase.
3. Record each successful meal handover exactly once.
4. Support both pickup and paid delivery without mixing delivery charges with subscription food entitlement.
5. Give providers a fast workflow for plans, bookings, QR scans, customers, earnings and settlements.
6. Give delivery partners a simple assignment/pickup/drop-off workflow with privacy boundaries.
7. Give admins operational oversight, disputes, verification and financial reconciliation.
8. Test provider adoption and renewals before optimising commission or expanding geographically.

4. Non-goals for MVP

- National multi-city operations at launch.
- In-house rider fleet employment or vehicle management.
- Complex route optimisation or guaranteed real-time GPS in every build.
- AI recommendations, calorie tracking, social features or meal reviews as launch blockers.
- Full accounting software for providers.
- Unlimited meal plans without daily caps.
- Cash settlement through a personal bank account or unverified manual payout workflow.
- Claiming a QR proves the customer physically ate the meal; it proves a recorded redemption/handover event.

5. User roles

Role	Needs	Primary surfaces
Customer	Find plans, pay, book, collect or receive food, track entitlements and resolve problems	Customer Flutter app + public/customer website

Role	Needs	Primary surfaces
Provider owner/manager	Onboard business, publish plans, manage slots, redeem meals, check earnings	Provider Flutter app + provider web portal
Provider counter staff	Scan and validate a meal quickly, without seeing unnecessary financial/customer data	Provider Flutter app, restricted permissions
Delivery partner	Accept work, see assignment details, confirm pickup/delivery, see payout	Delivery Flutter app
Admin/operations	Verify providers, resolve issues, configure commissions, reconcile payments, audit activity	Admin web portal
Support staff	Investigate assigned tickets with least privilege	Restricted admin route group

One person may hold multiple roles only if explicitly assigned by a trusted backend/admin process. Never infer roles from editable frontend profile metadata.

6. Core customer journeys

6.1 Discover and purchase

1. Customer enters location/address or selects a locality/pincode.
2. Browse eligible, active providers and their plans/menu.
3. View provider details, service hours, ratings only if real reviews exist, plan price, duration, meal slots, daily limits, holidays, pickup/delivery support and policies.
4. Select plan or a single meal.
5. Choose pickup or delivery where supported.
6. Review full price breakdown and terms.
7. Complete sandbox/live gateway checkout as configured.
8. Backend verifies payment before marking the purchase active/paid.
9. Show receipt and next steps.

6.2 Plan types

A. Fixed-period subscription

- Durations: 1, 3, 6 and 12 months (provider-configurable).
- Expires at a server-computed timestamp based on calendar-month arithmetic in the configured timezone. Interval is start-inclusive and expiry-exclusive.
- Meal count does not reduce a credit balance because the plan is time-based; however, daily/per-slot redemption limits still apply.
- Customer may eat zero or many eligible meals during the active period, subject to limits and provider schedule.

B. Meal-credit subscription

- Customer buys a defined number of meals (e.g. a configured 30-meal plan).
- A credit is consumed only after valid pickup redemption or successful delivery completion.
- A booking/reservation alone does not consume the credit.
- No-show/cancellation policy must be shown before purchase. MVP default: do not automatically deduct a credit merely because the customer did not arrive; record a missed/cancelled event and let policy rules govern exceptions.
- Prevent simultaneous bookings from overselling a scarce slot where capacity limits exist.

C. Daily single meal (pay-as-you-eat)

- Customer purchases a specific meal/menu item without a longer subscription.
- Price can be higher than equivalent subscription value.
- Payment must be verified before the order is treated as paid/accepted.
- Any included delivery fee is shown separately.

Do not label meal-credit tracking as GPS/live location tracking. It is meal usage tracking. Delivery location tracking is a separate feature.

6.3 Pickup redemption

1. Customer creates/opens an eligible booking.
2. Backend issues a short-lived, one-time QR token bound to the booking and provider.
3. Provider staff scans it while authenticated and authorised for that provider.
4. Backend validates customer, provider, slot/date, subscription/payment state, token expiry and prior redemption.
5. One atomic server transaction changes the booking to redeemed, inserts the redemption event and decreases meal credits if applicable.
6. Provider sees a clear success result and customer balance/history refreshes.
7. Replays, wrong-provider scans and concurrent duplicate scans fail without another deduction.

6.4 Delivery

1. Customer chooses delivery and an eligible address.
2. Backend checks provider coverage, availability, service hours and slot cutoff.
3. Checkout shows food/subscription amount and delivery charge separately.
4. A delivery order is created and payment for any due amount is verified.
5. A delivery partner is assigned/accepts the order.
6. Partner sees only information needed for the active assignment.
7. Partner confirms pickup, then completes drop-off using server-validated OTP/QR.
8. Backend atomically marks delivery complete and redeems the meal once.
9. Customer sees order status and receipt; provider sees completion and settlement ledger entry.

If a subscription covers the food, the customer must not be charged for the same food a second time. Delivery fee remains a separate payable item.

7. Provider capabilities

- Apply to join and submit business/contact/food licence details where applicable.
- Admin verification and activation.
- Manage business profile, address, operating hours, service areas and meal slots.
- Create/publish/pause meal plans and menu items.
- Set price, duration, meal credits, per-day/slot caps, holidays, booking cutoff, cancellation policy and delivery eligibility.
- View today's bookings and expected meals.
- Scan/validate QR codes and see success/failure reason.
- View active subscriptions, meal history, complaints and customer notes necessary for service.
- See gross sales, platform commission, refunds, net payable and settlement status as distinct values.
- View and accept explicit commercial agreement/commission terms before fee activation.

8. Delivery partner capabilities

- Apply and complete onboarding/verification required by the operating model.
- Online/offline availability toggle.
- View only eligible assignments.
- Accept/reject assignment, confirm pickup, update status and complete delivery.
- Read delivery instructions and navigate to drop-off.

- Access only the minimum customer contact/address data while assignment is active.
- View delivery history, earnings, adjustments, payout status and support options.
- Report failed delivery, unsafe address, customer unavailable or provider delay.

9. Website requirements

Public site: home, locality/provider discovery, provider profile, plan comparison, FAQ, support/contact, about, terms, privacy, cancellation/refund policy and food-safety/partner information as appropriate.

Customer portal: sign in, addresses, subscriptions, meal booking, QR/OTP, history, payments, delivery tracking/status, support and renewal.

Provider portal: onboarding, business profile, menu/plans, bookings/redemption activity, customers, delivery settings, earnings and settlements.

Admin portal: provider/rider approvals, user/support management, plan/content moderation, booking/redemption investigation, payment/refund/settlement views, commission rules, launch promotions and audit history.

All role portals must be protected server-side. Hiding a menu item in the UI is not authorization.

10. Commercial rules

- First 2 months: 0% provider commission promotional period, only under clear written/accepted terms.
- After the promotion: configurable 2–5% provider commission. Use 3% only as a test default if needed; do not bake it into code.
- Commission applies only to eligible transaction categories and terms explicitly agreed with each provider.
- Configure effective start/end timestamps, agreement version, accepted_at and rate in basis points.
- Delivery fee, rider payout and platform delivery fee are separate ledger values. A proposed 20% delivery-side commission is not final until rider willingness and per-trip economics are tested.
- Never deduct a provider commission twice or calculate commission from amounts that policy excludes (e.g. refunded amount, delivery fee if the provider commission is only on food).

11. MVP success metrics

Instrument these from the first pilot:

- Providers approved, active providers, and provider retention.
- Customer signups and first paid subscriptions.
- Paid conversion by provider/plan.
- First meal redemption rate and meals redeemed per active subscriber.
- Subscription renewal/retention by cohort.
- Failed/replayed QR scans and duplicate deduction count (target: zero duplicate deductions).
- Delivery acceptance, successful delivery, cancellation and incident rates.
- On-time meal readiness where measurable.
- Gross booking value, platform revenue, gateway cost, refunds, provider payable, rider payable and contribution margin.
- Support tickets per 100 bookings.

Do not optimise downloads alone; use completed paid subscriptions and repeat usage.

12. MVP acceptance criteria

- A real test user can discover a provider, view plan terms and complete a sandbox checkout.
- Payment success is confirmed server-side before activation.
- Fixed-period subscriptions expire on server-computed time and enforce daily/slot caps.
- Meal-credit purchases consume exactly one credit per successful meal redemption/delivery completion.
- Valid QR succeeds once; expired, replayed, wrong-provider and concurrent replay cases fail safely.
- Delivery charge is independently calculated and shown; subscription-covered food is not double charged.
- Provider and rider data boundaries are tested.
- Provider commission uses the agreed effective rule and produces auditable entries.
- Admin can review a payment, redemption, dispute and settlement trail.
- Critical flows work on mobile and web with loading, error, empty and success states.

02 — Technical Requirements Document (TRD)

1. Architecture decision

Use a monorepo unless the existing repository already has a sound alternative. Preserve existing working code and establish this target structure incrementally:

```
MESS/
  apps/
    customer_app/      # Flutter Android app
    provider_app/      # Flutter Android app
    delivery_app/      # Flutter Android app
    web/               # Next.js: public site + customer/provider/admin route groups
  packages/
    api_contracts/     # OpenAPI and generated/shared contract artifacts where practical
    design_tokens/     # Brand tokens and shared web components if appropriate
  supabase/
    migrations/
    functions/
    tests/
  docs/
  prompts/
  .env.example
  README.md
```

For Flutter, prefer shared Dart packages for models, API client, theming and common widgets while preserving role-specific entry points and permissions. If managing multiple Flutter packages requires extra tooling the repository does not have, do not introduce it without justification.

2. Applications

A. Customer Flutter app

Android first; responsive web provides complementary access. Screens: onboarding/auth, location/address, discovery, provider profile, menu/plans, plan details, checkout, subscription home, book meal, QR display, delivery status, history/receipts, support, profile/settings.

B. Provider Flutter app

Screens: provider onboarding/status, provider home, today's bookings, QR scanner, scanner result, plans, menu availability, subscriptions/customers, delivery requests/status, earnings/settlement summary, account/support. Role and provider membership must be resolved server-side.

C. Delivery Partner Flutter app

Screens: onboarding/verification, availability, assignment list/detail, accept/reject, navigation, pickup confirmation, drop-off verification, delivery history, payout summary, issue reporting. Do not add always-on background tracking to MVP unless explicitly approved and implemented with user permission, a visible purpose, retention limits and platform compliance.

D. Next.js web

A single web codebase can host:

- Marketing/public discovery pages.
- Customer portal under /account.
- Provider portal under /provider.
- Admin portal under /admin.

Use server-enforced access control for every protected page and API. Admin endpoints must not trust client role or user metadata. Provide separate navigation/layouts and strict feature access per role.

3. Backend

Preferred baseline: Supabase Auth, PostgreSQL, Storage and Edge Functions. Use server-side Edge Functions for privileged operations such as create checkout, verify payment/webhook, create QR token, redeem QR, confirm delivery, apply entitlements, make refunds/settlement ledger entries and admin operations. Current Supabase docs describe Edge Functions as server-side TypeScript endpoints and recommend environment-stored secrets; validate exact CLI/runtime guidance during implementation.

Do not make every screen call service-role-powered endpoints; use the minimum privileged capability for each operation. Non-sensitive owner-scoped reads can use Supabase client + RLS where appropriate. For sensitive multi-step business operations, expose a narrowly scoped server endpoint and perform the required database writes atomically.

4. Contracts and consistency

- API contracts live in docs/07_API_CONTRACT.md or an OpenAPI file; update them when endpoints change.
- Store generated database types; avoid handwritten mismatched database models.
- Money is integer paise (*_paise), not floating point rupees.
- Timestamp storage is timestamptz (UTC at storage); render in Asia/Kolkata and the provider's configured timezone.
- Calendar plan expiry must be computed server-side with timezone-aware calendar arithmetic. Use an expiry-exclusive interval [starts_at, expires_at).
- Use UUID identifiers for entities. Do not expose sequential internal financial IDs.
- All create/payment/redemption/delivery-completion endpoints should support idempotency.
- Financial entries are append-only; correct errors with reversal/adjustment entries instead of silently rewriting history.

5. Auth and authorisation

- Supabase Auth is the identity provider.
- A user_roles table stores trusted role assignments. Clients have no direct insert/update/delete rights on role assignments.
- provider_memberships maps provider users to provider-specific roles (owner, manager, counter_staff).
- Delivery partner access requires a verified partner profile and assignment ownership.
- Admin access is assigned by a trusted admin process and enforced server-side.
- Treat raw_user_meta_data/client-editable user metadata as untrusted.
- Enable RLS on every exposed table. Configure explicit grants and per-operation policies; RLS policies alone do not remove grants.
- Test allowed and denied access for anon, customer, provider, rider and admin flows.

6. Payment integration

Build an interface such as PaymentGatewayAdapter with operations for create checkout/order, verify server callback/webhook, refund, and fetch transaction state. Integrate one selected Indian gateway in sandbox first. Do not hardcode assumptions about marketplace settlement availability or gateway APIs.

Rules:

- Never activate an entitlement based only on a frontend return URL.
- Verify the gateway's signature using server-held secrets and use current official docs.
- Store a gateway order/payment reference and idempotency key.
- Process webhooks idempotently using a unique provider event ID.
- Reconcile payment state against gateway state when webhooks are delayed or duplicated.
- Store only required, sanitised gateway payload fields; never store card credentials or secret values.
- Keep checkout total server-calculated. Ignore client-supplied totals.

- Live settlement remains blocked until gateway onboarding, KYC, settlement design and compliance checks are complete.

7. QR and redemption

- Backend generates a cryptographically unpredictable token; store only a hash or otherwise protected representation.
- Token has one booking, one provider, a purpose, issue time and expiry.
- Return raw token only when generated; avoid logging it.
- Server verifies token, booking state, service date/slot, customer/provider binding and payment/subscription eligibility.
- Redeem with a transaction/row lock/conditional update so concurrent scans cannot both win.
- Unique constraint on meal_redemptions.booking_id ensures one successful redemption per booking.
- A failed attempt does not deduct credits. Rate-limit repeated invalid scans.
- Refresh tokens when expired, but do not create a new eligible booking or reset the prior redemption state.

8. Delivery

- MVP uses zone/pincode/radius eligibility; add map/routing provider only when configured.
- Delivery price is computed on the server from provider settings, zone/distance inputs and pricing rules.
- Each delivery assignment belongs to exactly one booking and one accepted partner at a time.
- Use legal state transitions: offered -> accepted -> picked_up -> delivered, with explicit rejected, cancelled, failed branches.
- Only the currently assigned partner or authorised operations endpoint may update the assignment.
- Location events are optional; do not fake live tracking. If enabled, collect only during active delivery and define retention.
- Delivery completion and meal redemption are one idempotent transaction/event chain.

9. Observability and reliability

- Structured logs with request/correlation ID; no passwords, raw QR tokens, payment secrets or unnecessary personal data.
- Error tracking and uptime monitoring configurable by environment.
- Record domain audit events for role changes, provider approvals, commission acceptance, refunds, manual adjustments and overrides.
- Retry transient gateway/network calls with bounded exponential backoff and idempotency.
- Use background jobs/cron for expiry reminders, subscription expiry state maintenance, unpaid checkout cleanup and settlement reconciliation. Never rely only on a client timer to expire plans.
- Avoid unbounded Realtime subscriptions; scope subscriptions to the signed-in user and active workflow.

10. Performance and usability targets (initial engineering targets)

- Typical read endpoints: p95 under 700 ms under pilot-scale load, excluding external gateway/network latency.
- QR validation and redemption: target p95 under 1 second under pilot-scale load; correctness beats speed.
- Avoid N+1 queries and index foreign keys and columns used in RLS filters/state queries.
- All core flows usable on mid-range Android phones and intermittent mobile networks.
- Show offline/network error states. Do not permit an offline QR scan to claim success without server validation.

These are initial engineering targets, not guarantees; measure and tune after the first pilot.

11. Environment strategy

- local/development, staging, production must be separated.
- Different Supabase projects/credentials for each environment where possible.

- .env.example documents names only; never overwrite .env or commit secrets.
- Sandbox payment mode by default.
- Production deployments, migrations, live payments, money movement and real customer notifications require explicit approval.
- Seed fake development data only; never put fake transaction/settlement entries in production.

12. Deployment target

- Web: deploy to a supported Next.js host after build and environment checks.
- Supabase: migrations and Edge Functions deployed through documented, reproducible workflow.
- Flutter: debug/release builds and Android APK/AAB generated only after analysis and tests pass.
- CI should run formatting, lint, type checks, unit tests, database tests and web build. Add Flutter analysis/tests for all three apps.
- Add rollback/forward-fix instructions for database and web releases.

13. Required technical deliverables

- Architecture and data-flow diagram.
- Environment-variable inventory and .env.example.
- Versioned SQL migrations.
- RLS/grant policies and database tests.
- API contracts / OpenAPI where practical.
- Three Flutter Android projects.
- Web portal route groups and server authorisation.
- Payment gateway adapter in sandbox.
- QR issue/redeem logic with replay/concurrency tests.
- Delivery state machine and idempotent completion.
- Commission/ledger/reconciliation logic.
- CI/check scripts, test reports, deployment runbook and known-limitations list.

03 — App and Website Flows

1. System-level journey

```
flowchart TD
  A[Customer visits MESS app or website] --> B[Choose location / locality]
  B --> C[Browse active providers]
  C --> D[Provider profile, menu and plan rules]
  D --> E{Purchase type}
  E --> F[Fixed-period plan]
  E --> G[Meal-credit plan]
  E --> H[Single meal]
  F --> I[Server creates pending checkout]
  G --> I
  H --> I
  I --> J[Payment gateway checkout]
  J --> K{Verified server-side?}
  K -- No / pending --> L[Show pending or failure; no entitlement yet]
  K -- Yes --> M[Activate subscription or mark meal paid]
  M --> N{Pickup or delivery?}
  N --> O[Pickup booking and QR]
  N --> P[Delivery booking and separate delivery fee]
  O --> Q[Provider scans QR]
  Q --> R[Atomic redemption + ledger update]
  P --> S[Assign delivery partner]
  S --> T[Pickup and drop-off confirmation]
  T --> U[Atomic delivery completion + redemption]
  R --> V[Update customer history and provider reporting]
  U --> V
```

2. Customer mobile app flow

1. Splash → onboarding/permissions only when necessary.
2. Sign up/sign in → profile → location permission or manual locality/address.
3. Home → search/filter → provider list → provider detail.
4. Provider detail → plans/menu → plan policy screen.
5. Choose plan and intended fulfillment preference (provider may require pickup or allow delivery).
6. Checkout shows item/plan price, discount, delivery fee if any, taxes, total, refund/cancellation policy, plan expiry and included meal rules.
7. Payment gateway → server verification → success/pending/failure screen.
8. Active plan screen displays status, start/expiry timestamp, fixed-period eligibility or remaining credits, daily/slot limits and next actions.
9. Book meal → choose date/slot → pickup/delivery → confirm booking.
10. Pickup: open QR → provider scans → success state refreshes.
11. Delivery: address and delivery quote → pay any delivery fee → order tracking timeline → secure drop-off confirmation → completed state.
12. History/receipts/support/renewal/profile.

Error paths: payment pending, expired plan, no capacity, provider closed, slot cutoff passed, out-of-service-area address, QR expired, network failure, delivery partner unavailable and refund requested. All paths need explicit recovery.

3. Customer website flow

Public routes

- / home and locality search.
- /explore provider discovery.
- /providers/[slug] provider details, menu and plans.

- /plans/[id] optionally dedicated plan detail.
- /how-it-works, /faq, /contact, /about.
- /terms, /privacy, /cancellation-refunds.

Authenticated customer routes

- /account overview.
- /account/subscriptions and /account/subscriptions/[id].
- /account/bookings and /account/bookings/[id].
- /checkout/[sessionId] (server-created checkout session; never accepts trusted amounts from URL).
- /account/addresses, /account/payments, /account/support, /account/settings.

The website may display a QR only for the authenticated owner of the eligible booking. If QR scanning is app-only initially, website should deep-link to the customer app or show a safe booking reference; do not create an unprotected QR URL.

4. Provider app flow

1. Sign in → check role, membership and provider approval status.
2. If no provider is assigned: application onboarding and document upload.
3. Pending approval screen until admin approves.
4. Provider home: today's slots, booking counts, meals ready, redemption activity and current incidents.
5. Scan QR → permission request → scan result.
6. Backend result: valid/invalid/already used/expired/wrong provider/unpaid/ineligible plan/capacity or schedule mismatch.
7. On success show customer initials/reference, meal slot and exact result; avoid unnecessary personal data.
8. Plan/menu management → draft → preview → publish/pause.
9. Booking list → ready/closed controls where allowed.
10. Delivery requests → accept/mark food ready/handover to assigned partner.
11. Earnings → commission and refunds → net payable → settlement status.
12. Support and settings.

Counter staff may scan/redeem but cannot edit bank/payout details or commission terms. Only provider owner/manager can change authorised business settings.

5. Provider website portal flow

- /provider/onboarding
- /provider/overview
- /provider/profile
- /provider/menu
- /provider/plans
- /provider/slots-hours
- /provider/bookings
- /provider/redemptions
- /provider/customers
- /provider/delivery
- /provider/earnings
- /provider/settlements
- /provider/team
- /provider/support

All route data must be scoped to the current provider membership. A user belonging to Provider A must not read or modify Provider B by changing an ID in the URL.

6. Delivery app flow

1. Sign in → partner verification status.
2. Pending/approved states.
3. Go online → receive eligible offered assignment.
4. Review pickup area, drop-off locality, estimated distance/payout and required timing. Do not expose full address before assignment acceptance unless operationally required.
5. Accept/reject → backend checks partner availability, assignment status and idempotency.
6. Navigate → arrive at provider → confirm pickup.
7. Deliver to customer → collect/validate OTP or delivery QR.
8. Server validates assigned partner, order, OTP expiry, booking state and duplicate completion.
9. Mark delivered; backend records completion and, if applicable, meal redemption/credit deduction.
10. Delivery history, payout ledger, issues/support.

If the partner has no network, allow only a clearly marked retry/pending state. Never show delivery completed until server verification succeeds.

7. Admin website flow

1. Admin authenticates with protected access.
2. Overview: pending providers/riders, unresolved tickets, disputed payments, active deliveries and payout/reconciliation exceptions.
3. Review provider application/document status → approve/reject/request correction with audit note.
4. Review delivery partner eligibility.
5. Search users/provider/subscription/booking/payment by safe identifiers.
6. Investigate QR/meal redemption without exposing raw tokens.
7. Approve or record a refund through supported workflow; never manually mutate a gateway payment state.
8. Configure platform promotion and commission rules with effective dates.
9. Ensure provider accepted terms before new commission becomes effective.
10. Review settlement ledger and reconciliation.
11. Manage support ticket and issue resolution.
12. Inspect audit trail.

Admin overrides require explicit reason, permission and audit log. Financial corrections should be reversal/adjustment ledger entries, not silent edits.

8. State machines

Subscription

pending_payment → active → paused (optional) → active → expired

Additional terminal/exception states: cancelled, refund_pending, refunded, payment_failed.

- Only verified payment makes a paid subscription active.
- Fixed-period expiry is server-derived with exclusive expires_at.
- Pause/extension rules must be explicit and auditable; do not change expiry from the client.

Meal booking

draft → pending_payment (single meal or delivery fee) → confirmed → ready → redeemed for pickup.

Delivery: draft → pending_payment → confirmed → assigned → accepted → picked_up → delivered.

Other allowed outcomes: cancelled, missed, failed_delivery, refund_pending, refunded.

A booking may be redeemed only once. Delivery completion and entitlement deduction are idempotent. A mere transition to ready does not redeem a meal.

Delivery assignment

offered → accepted → picked_up → delivered; exceptions include rejected, cancelled, failed, reassigned. Reassignment must invalidate previous partner update authority.

Payment

created → pending → succeeded or failed; a successful transaction may later be associated with a separate refund record. Do not overwrite a succeeded payment to failed because of a late duplicate webhook.

9. Design/interaction principles

- Always show what is included in a plan and whether delivery costs extra.
- Never show a successful scan/payment/delivery until backend confirmation.
- Use meaningful empty/error/loading states.
- Confirm irreversible actions such as cancel/refund/commission changes.
- Maintain accessibility, readable contrast and low-friction interaction on mid-range Android devices.

04 — Backend Schema and Data Rules

The corresponding proposed SQL migration is at supabase/migrations/20261024000100_mess_core_schema.sql. Review and run it only after comparing it with the actual repository, current Supabase/Postgres version and test environment.

1. Schema principles

- PostgreSQL is the source of truth for roles, entitlements, booking/payment state, redemption and financial ledger.
- All user-facing entities use UUID primary keys.
- Money is integer paise; display as INR only in clients.
- All stored timestamps are timestamptz. Store service-day separately where local meal dates matter.
- Historical pricing/terms on a transaction are snapshots; do not retroactively change old booking totals when a plan price changes.
- Use foreign keys, CHECK constraints, unique constraints and appropriate indexes.
- Financial activity is auditable and effectively append-only; corrections use reversals/adjustments.
- Apply RLS and explicit grants for every exposed table; no broad authenticated can do everything policy.

2. Main entities

Entity	Purpose / important fields
profiles	Application profile for an auth user; no client-editable role field
user_roles	Trusted role assignments: customer, provider, delivery partner, admin, support
customer_addresses	Customer-owned saved addresses and delivery instructions
providers	Business profile, owner/status, timezone, contact, food licence/reference and approval state
provider_memberships	Maps user to provider with owner/manager/counter-staff permissions
provider_service_areas	Pincode/locality/zone or radius used for discovery and delivery eligibility
provider_operating_hours	Weekly schedule and closure windows
meal_slots	Lunch/dinner/etc. windows, cutoffs, capacity and active state
menu_items	Provider-owned meals/combos, description, price and availability
subscription_plans	Fixed-period or meal-credit plans, price, duration/credits, terms and status
plan_meal_slots	Slot eligibility for a plan
subscriptions	Customer purchase snapshot, start/expiry/status, initial/remaining credits
meal_bookings	One meal booking for a subscription or single-meal purchase; pickup/delivery and price snapshot
qr_redemption_tokens	Hashed, purpose-bound, short-lived one-time token per booking
meal_redemptions	Immutable record of successful meal redemption/handover; unique booking

Entity	Purpose / important fields
delivery_partner_profiles	Partner verification/availability and payout onboarding status
delivery_assignments	Booking-to-partner relationship and legal delivery state transitions
delivery_location_events	Optional short-retention delivery location samples; only when tracking is enabled
payment_transactions	Gateway order/payment reference, purpose, amount, state and idempotency key
payment_webhook_events	Gateway event ID, verification state, processing outcome and sanitized metadata
refunds	Separate refund records tied to a payment transaction
platform_commission_rules	Effective-dated, configurable basis-point rules by transaction category/provider
commercial_agreements	Accepted commission/promotion terms, version, rate and dates
financial_ledger_entries	Provider/platform/rider/gateway financial event ledger
provider_settlements	Period totals and status for provider settlements
provider_settlement_items	Ledger entries included in a settlement
delivery_payouts	Delivery partner payout/adjustment records
notifications	In-app notifications and read status
support_tickets	Customer/provider/rider issues, assigned staff and resolution
audit_logs	Sensitive actions and overrides, actor, subject, reason and timestamp
idempotency_records	Durable idempotency keys for critical operations where needed

3. Core relational rules

- A provider owner/manager/counter staff record must reference one provider and a trusted user.
- A subscription must reference the provider and plan that were purchased, and should retain price/terms snapshots.
- fixed_period subscriptions have start/expiry timestamps; meal-credit plans also have initial/remaining credits.
- A meal booking belongs to one customer and one provider; it may reference a subscription or be a single-meal purchase, never both without explicit modelling.
- Delivery requires a valid delivery address snapshot and a delivery fee. Pickup does not have a delivery fee.
- A QR token has a single booking, provider and purpose; raw tokens are never stored in a table or logged.
- meal_redemptions.booking_id is unique. A booking cannot be redeemed twice.
- Each payment webhook provider event ID is unique; duplicate webhooks must return the prior processing result.
- A delivery assignment must refer to a valid booking and an authorised delivery partner.
- Commission rule is selected by effective date and transaction category; applied rate and base amount are copied into ledger/settlement records.
- Refund entries never erase original payment or ledger records.

4. Money and commission

Use BIGINT paise values for all amounts. Use integer basis points for percentages: 2% = 200 bps, 3% = 300 bps, 5% = 500 bps, 20% = 2000 bps. Round according to a single documented server-side rule; record calculated commission base and amount for audit.

Separate at minimum:

- Food/plan amount.
- Amount covered by an existing subscription (not charged a second time).
- Discount.
- Tax if applicable and configured by qualified advice.
- Delivery charge.
- Gateway fee (as reported by gateway where available).
- Provider commission.
- Platform delivery fee/commission.
- Rider payout.
- Refund/reversal.
- Provider net payable and rider net payable.

Do not assume all these amounts are known at checkout; estimates and final reconciliation may differ. Clearly label estimates. Never use a client-provided amount as authoritative.

5. Transaction boundaries

Payment verification

1. Verify gateway callback/webhook signature.
2. Insert or claim unique event/idempotency key.
3. Fetch/verify authoritative gateway order/payment state.
4. Lock the associated pending transaction/booking/subscription.
5. Apply state change once; activate subscription or mark booking paid.
6. Insert financial ledger entry/commission calculation as appropriate.
7. Mark webhook processed and return success.

Pickup redemption

1. Hash incoming token and locate unexpired unconsumed token.
2. Lock token and booking.
3. Verify actor membership for the exact provider.
4. Verify booking/subscription/slot/payment eligibility.
5. Insert one redemption record with unique booking ID.
6. Decrease one meal credit if applicable, with a guard against balance < 0.
7. Mark token used and booking redeemed.
8. Commit as one transaction.

Delivery completion

1. Verify authenticated partner and exact active assignment.
2. Verify valid one-time OTP/QR, assignment status and booking eligibility.
3. Lock assignment and booking.
4. Insert delivery completion event and redemption once.
5. Decrease meal credit if applicable.
6. Write rider/provider/platform ledger entries according to agreed rule.
7. Commit atomically; duplicate request returns prior result.

6. RLS matrix (policy intent)

Data	Public/anon	Customer	Provider staff	Delivery partner	Admin/support
Active public provider/menu/plan	Read public fields only	Read	Own provider + public	Public fields only	Server-authorized
Own profile/addresses	None	Own rows	Own profile	Own profile	Restricted support/admin
Subscriptions	None	Own rows	Own provider subscriptions, minimal fields	None	Server-authorized
Bookings	None	Own rows	Own provider bookings	Only active assigned bookings, minimal fields	Server-authorized
QR tokens	None	No raw stored token access; get token through endpoint	Validate through endpoint, no bulk token read	Validate delivery token through endpoint	No raw token access; audit metadata only
Redemptions	None	Own history	Own provider history	Only assigned delivery history	Server-authorized
Payments/ledger/settlements	None	Own transaction/receipt fields	Own provider summarized finance/settlements	Own payout summaries	Restricted server-authorized access
Roles/memberships	None	Read own role/membership as needed	Read own provider membership	Read own verification state	Mutations via admin service only
Audit/webhook/raw gateway events	None	None	None	None	Restricted server-side only

This is an intent matrix, not a substitute for executable policies. Build supabase/tests tests for allowed and denied cases across all roles. Do not enable production until RLS/grant audit is clean.

7. Schema evolution rules

- Use new timestamped migration files; do not hand-edit already-applied production migrations.
- Test migrations on a development/staging database first.
- Avoid destructive migrations without explicit approval, backup and rollback/forward-fix plan.
- Re-generate client types after schema changes.
- Add tests for new constraints, grants, RLS and transaction logic.

07 — API Contract Responsibilities (MVP)

This is a domain-level contract, not a claim that routes already exist. Antigravity should implement typed request/response schemas and preferably maintain an OpenAPI 3.1 file matching the implemented endpoints. All paths below are illustrative; preserve consistency once selected.

API conventions

- JSON over HTTPS for application APIs.
- Use standard status codes and structured errors such as `{ "code": "QR_ALREADY_REDEEMED", "message": "This meal has already been redeemed", "request_id": "..." }`.
- Require an authenticated session for non-public routes.
- Include `Idempotency-Key` for create checkout, booking, refund, QR redemption and delivery completion where appropriate.
- Server-derived values override client values.
- Use pagination for list endpoints.
- Never return raw gateway secrets, raw token hashes, other customers' personal details, or internal stack traces.

Customer / discovery

- `GET /providers?locality=&pincode=&pickup=&delivery=&q=` — active public providers only.
- `GET /providers/{providerId}` — public profile, schedules, published plans/menu, supported service modes.
- `GET /providers/{providerId}/plans` — published plans and clear policy summaries.
- `GET /me` — own profile and permitted role list.
- `GET /me/addresses`; `POST /me/addresses`; `PATCH /me/addresses/{id}`; `DELETE /me/addresses/{id}` — own addresses only.

Checkout / subscription

- `POST /checkout/subscriptions` — input: plan ID, selected duration if plan supports it, idempotency key. Server reads price and terms, creates pending subscription/payment transaction and returns checkout instructions.
- `POST /checkout/single-meal` — input: provider ID, menu item ID, slot/date, fulfillment mode, optional address ID and idempotency key. Server validates availability and calculates the total, including any amount covered by a subscription so the same food is never charged twice.
- `POST /payments/{gateway}/webhook` — signature-verified gateway event; idempotent event processing.
- `GET /payments/{transactionId}` — caller sees only permitted fields; reconcile pending state when appropriate.
- `POST /subscriptions/{id}/pause` and `/resume` — only if pause policy allows; server derives eligibility and expiry updates.
- `GET /me/subscriptions`; `GET /me/subscriptions/{id}` — own subscription and entitlement state.

Meal booking / QR

- `POST /meal-bookings` — input: subscription ID or single meal reference, date, slot, pickup/delivery, optional address ID, idempotency key. Server validates schedule/capacity and does not deduct credit.
- `GET /me/meal-bookings` — customer-owned list.
- `POST /meal-bookings/{id}/qr` — only booking owner can request a short-lived pickup QR; ensure booking is eligible and state permits issue.
- `POST /redemptions/validate-and-redeem` — authenticated provider staff submits token; server resolves exact provider from staff membership, validates all rules and atomically redeems once. Do not trust provider ID sent by client.
- `GET /providers/me/redemptions` — provider-scoped redemption history, paginated.

Delivery

- POST /delivery/quote — booking/address/provider data; returns server-calculated fee, estimated zone/range if supported and quote expiry.
- POST /delivery/assignments/{id}/accept — verified eligible partner accepts if assignment is still open.
- POST /delivery/assignments/{id}/reject — reject with reason code.
- POST /delivery/assignments/{id}/pickup — current assigned partner confirms pickup.
- POST /delivery/assignments/{id}/complete — current assigned partner submits OTP/QR; server completes once and redeems the meal atomically.
- POST /delivery/assignments/{id}/report-issue — issue category and safe details.
- GET /delivery/me/assignments; GET /delivery/me/earnings — own assignments and payout summaries only.

Provider operations

- POST /provider/applications; GET /provider/me.
- PATCH /provider/me/profile; PATCH /provider/me/hours.
- POST /provider/me/menu-items; PATCH /provider/me/menu-items/{id}; POST /provider/me/plans; PATCH /provider/me/plans/{id}.
- GET /provider/me/bookings?date=&slot=&status= — provider-scoped.
- POST /provider/me/bookings/{id}/ready — only allowed state transition.
- GET /provider/me/settlements; GET /provider/me/financial-summary — provider scoped and no gateway secrets.
- POST /provider/me/commercial-agreement/accept — records explicit version/rate/effective terms, with server-derived identity and timestamp.

Admin/operations

Admin routes are server-protected and need stronger audit logging:

- GET /admin/providers?status=; POST /admin/providers/{id}/approve; POST /admin/providers/{id}/reject.
- GET /admin/delivery-partners?status=; verification actions.
- GET /admin/transactions; GET /admin/bookings/{id}/audit.
- POST /admin/refunds — only after verifying underlying gateway/refund policy; idempotent.
- POST /admin/commission-rules; POST /admin/promotions.
- GET /admin/settlements/reconciliation; GET /admin/audit-logs.
- Support ticket assignment/resolution endpoints.

Error codes to standardise

UNAUTHENTICATED, FORBIDDEN, NOT_FOUND, VALIDATION_ERROR, PROVIDER_INACTIVE, PLAN_UNAVAILABLE, SUBSCRIPTION_INACTIVE, SUBSCRIPTION_EXPIRED, MEAL_CREDIT_EXHAUSTED, SLOT_UNAVAILABLE, BOOKING_CUTOFF_PASSED, OUT_OF_DELIVERY_AREA, PAYMENT_PENDING, PAYMENT_FAILED, PAYMENT_NOT_VERIFIED, QR_EXPIRED, QR_INVALID, QR_ALREADY_REDEEMED, WRONG_PROVIDER, ASSIGNMENT_NOT_ACTIVE, DELIVERY_OTP_INVALID, IDEMPOTENCY_CONFLICT, RATE_LIMITED, INTERNAL_ERROR.

Required endpoint tests

For every endpoint, test valid request, invalid payload, unauthenticated request, wrong-role request, cross-tenant ID substitution, repeated idempotency key, concurrent request and relevant external provider failure.

05 — Implementation Plan and Verification Gates

1. Build strategy

Build in vertically complete slices, not as a large set of disconnected screens. The target is three Flutter apps plus a website/portals, but the first priority is one complete customer-to-provider transaction end-to-end. Keep a living checklist and loop after every phase: inspect → implement → run checks → exercise flow → fix → rerun → mark verified.

2. Phase plan

Phase 0 — Repository discovery and decisions

Tasks

- Inspect repo, existing app(s), package manager, Flutter/Node versions and existing design.
- Check environment files without printing secrets; record missing configuration variable names only.
- Map implemented vs missing requirements.
- Decide monorepo layout based on existing work; avoid destructive rewrite.
- Create docs/DECISIONS.md, task checklist and local run instructions.

Exit gate: architecture plan and repo inventory exist; no working code/config deleted.

Phase 1 — Foundations and design system

- Set brand tokens for MESS (navy, orange/gold, warm cream, white, restrained green).
- Set responsive web shell and separate route groups.
- Build shared Flutter UI/API/model packages where useful.
- Set environment validation and .env.example.
- Configure lint, formatting, type checks, Flutter analyzer and test commands.

Exit gate: web build and each Flutter project scaffold can run; no fake “production-ready” integration claims.

Phase 2 — Database, auth and security first

- Review proposed SQL and create versioned migrations using actual Supabase CLI workflow.
- Add tables, constraints, indexes, grants and RLS.
- Implement profile and trusted role assignment.
- Implement provider membership isolation and rider assignment rules.
- Add policy tests for allowed and denied access.

Exit gate: DB migrations apply to dev cleanly; RLS tests run and pass for core data boundaries.

Phase 3 — Provider onboarding and public discovery

- Provider apply/edit profile, service area, opening hours, slots, menu and plan editor.
- Provider status/approval workflow.
- Public website discovery, provider detail, plans and transparent policy display.
- Seed development-only test providers.

Exit gate: approved provider can publish a plan; public site shows only active/published eligible provider content.

Phase 4 — Customer purchase + payment sandbox

- Customer auth/address.
- Plan detail/checkout and single-meal checkout.
- Gateway adapter and sandbox checkout creation.
- Signed callback/webhook verification and idempotency.

- Purchase receipt, pending/failed/success states.

Exit gate: subscription stays inactive before verified success; duplicate callback doesn't duplicate activation or ledger entries.

Phase 5 — Meal booking, QR redemption and balances

- Fixed-period subscription expiry and per-day/slot limits.
- Meal-credit entitlement/ledger logic.
- Meal booking calendar/cutoffs/capacity.
- QR issuance, scanner, result screen and redemption transaction.
- History and provider today's bookings.

Exit gate: valid scan redeems once; replay, expiry, wrong provider and concurrent scans fail without duplicate deduction.

Phase 6 — Delivery and payouts/fees

- Delivery zone/pincode support and fee quote.
- Customer delivery checkout with separate food/delivery amounts.
- Delivery partner onboarding/approval/availability.
- Assignment accept/reject, pickup, OTP/QR drop-off and status timeline.
- Atomic delivery completion + meal redemption.
- Rider/provider/platform ledger entries; payout reporting.

Exit gate: no delivery marked delivered without server verification; credits are not double-deducted; out-of-zone cases are rejected.

Phase 7 — Finance, commissions and admin

- Configurable provider commission and 2-month promo agreements.
- Terms acceptance and effective date handling.
- Financial ledger, refund records, provider settlements and rider payouts.
- Admin verification, ticket triage, dispute/search tools and audit logs.

Exit gate: test transactions produce correct expected ledger/commission/settlement totals, including refunds and promotion expiry.

Phase 8 — Complete all app and web surfaces

- Wire customer app, provider app and delivery app to already tested contracts.
- Complete mobile UX, push/in-app notification adapter and retry/loading states.
- Complete public website, customer portal, provider portal and admin portal.
- Avoid rebuilding business logic separately in each client.

Exit gate: same critical rules behave consistently across app and web; role access tests pass.

Phase 9 — Hardening, QA and pilot

- Unit/integration/E2E/security tests.
- Build and run all Flutter apps, web build and Supabase tests.
- Check mobile layouts and web routes.
- Add crash/error monitoring and backups/runbook.
- Test with fake/sandbox data and small controlled provider pilot.
- Measure metrics in PRD; fix highest-impact problems.

Exit gate: no known critical/high security bugs, core acceptance criteria pass, external blockers documented, production launch explicitly approved.

3. Task-tracking format

Every task must have:

- ID and phase.
- Description.
- Dependencies.
- Files/routes/database changes.
- Test command(s).
- Status: NOT STARTED, IN PROGRESS, BLOCKED, VERIFIED.
- Evidence of verification (actual command result / test case).

Do not mark VERIFIED because code was written. It must have been run and checked.

4. Continuous loop prompt

After each phase, Antigravity should:

1. Inspect repository diffs and runtime state.
2. Compare implementation against PRD/TRD/flow and schema docs.
3. Select the highest-priority unfinished item or failure.
4. Implement only a coherent slice.
5. Run lint/type/analyzer/tests/build appropriate to changed code.
6. Exercise the user flow manually/browser automation if available.
7. Review security and error paths.
8. Fix failures and rerun affected checks.
9. Update checklist, decisions and remaining blockers.
10. Continue without repeating work already verified.

After 2–3 failed attempts on the same issue, inspect logs/docs and change the approach; do not repeat a failing command indefinitely.

5. Priority order when time/context is limited

P0: auth/role boundaries, payments, entitlements, QR replay prevention, delivery completion, RLS and money ledger.

P1: provider onboarding, plan management, customer discovery/checkout, history and admin verification.

P2: notifications, refined analytics, map/live tracking and aesthetic polish.

Never trade P0 correctness for a more polished UI.

6. Suggested demo script

1. Sign in as a seeded customer and browse active provider.
2. Buy a meal-credit plan with sandbox payment.
3. Confirm no activation before server payment verification.
4. Book a pickup meal and show QR.
5. Sign in as provider counter staff and scan it.
6. Confirm exactly one credit is consumed and history updates.
7. Replay the same QR and confirm rejection.
8. Create a second booking for delivery and pay delivery fee in sandbox.
9. Assign a seeded rider, accept, mark picked up and complete OTP.
10. Confirm single meal redemption, delivery status and ledger entries.
11. Sign in as provider owner and view commission/payout summary.
12. Sign in as admin and inspect audit trail.

06 — Security, Testing, Operations and Release Readiness

1. Security baseline

- Enable RLS on all exposed tables; pair policies with explicit grants/revokes.
- Do not ship service-role/secret keys in Flutter, browser bundles, public environment variables or repository history.
- Role assignment and admin access are server-controlled.
- Enforce ownership, provider membership and delivery assignment on the backend and in database policies.
- Validate all request payloads; do not trust client prices, commissions, role claims, provider IDs, credits or booking status.
- Apply rate limits to login/OTP, QR validation, webhook and checkout endpoints.
- Use secure randomness; hash short-lived tokens; redact raw tokens from logs.
- Verify webhook signatures against raw request bodies as required by the provider's current official docs.
- Store only sanitised gateway payloads and minimum necessary customer information.
- Protect address/contact details; restrict data to active operational need.
- Upload provider documents to private storage buckets with limited signed access and appropriate retention.
- Prevent IDOR/BOLA by checking authorization at every resource operation, not just route navigation.
- Admin overrides need permission, reason and immutable audit entry.
- Define secure password/OTP, session expiry and rate-limit settings based on provider capabilities.

2. Critical tests

Authentication and RLS

- Anonymous user cannot read customer/private data.
- Customer A cannot read Customer B's address, subscription, payment or QR booking.
- Provider A cannot read/edit Provider B's menu, booking, membership or settlement.
- Counter staff can scan but cannot edit bank/payout settings or commission terms.
- Rider A cannot see/advance Rider B's assignment.
- Client cannot grant itself admin/provider/rider role.
- Admin-only actions fail for non-admin even if endpoint is called directly.

Payment

- Checkout total is calculated server-side.
- Failed, pending or forged frontend callback does not activate plan.
- Invalid signature is rejected.
- Duplicate webhook does not duplicate activation, commission or ledger event.
- Out-of-order webhook cannot regress a successful transaction to failed.
- Refund is a separate recorded event and reverses appropriate amounts according to policy.
- Payment provider timeout has a safe pending/reconciliation path.

Subscription and redemption

- Fixed-period validity and expiry-exclusive boundary.
- Month arithmetic around short months and timezone transitions.
- Daily/slot cap enforced under concurrent booking attempts.
- Meal credit does not go negative.
- Booking alone does not consume credit.

- One valid pickup scan consumes one credit exactly once.
- Expired/replayed/wrong-provider/wrong-booking QR fails.
- Concurrent requests using same token yield one success only.
- Delivery completion and redemption do not double-consume a credit.
- Provider closures, cancellation cutoffs and failed delivery follow explicit policy.

Delivery

- Out-of-zone delivery is rejected.
- Partner must be approved/online/eligible to accept an assignment.
- Only assigned partner can confirm pickup/drop-off.
- Old partner cannot complete reassigned assignment.
- Incorrect/expired OTP fails without completing delivery.
- Duplicate completion returns existing result and makes no duplicate ledger entries.
- Rider payout equals configured formula; commission is not assumed/ hardcoded.

Finance/commission

- 0% promotional period applies only within agreed effective dates.
- Post-promotion rate activates only after agreement acceptance and effective time.
- Provider commission base excludes delivery fee where terms say so.
- Refund/reversal calculations preserve audit history.
- Ledger and settlement totals reconcile to payment/booking source records.
- Paise arithmetic/rounding is deterministic.

UX/quality

- No dead buttons, fake charts, placeholder links or fabricated success states.
- Loading/empty/error/retry states exist for primary routes.
- Web responsive checks and keyboard/accessibility checks.
- Flutter analyzer/test/build pass independently for each app.
- API/database errors are useful to logs but not leaking internals to users.

3. Observability

Log structured events with request ID and entity IDs where safe. Exclude secrets, payment signatures, raw QR/OTP, full addresses and unnecessary personal data. Monitor error rate, payment pending duration, webhook failures, QR rejects/replays, redemption latency, delivery incidents, settlement mismatch and app crashes.

Create alerts for repeated payment verification failure, duplicate ledger attempts, unusual invalid QR volume, unreconciled settlements and delivery assignment failures.

4. Backups and recovery

- Confirm database backup/restore capabilities for the chosen Supabase plan.
- Document who may run migrations, deploy functions and release mobile apps.
- Test restore or recovery procedure in a non-production environment.
- Keep migration history and deployment versions in source control.
- Prefer forward-fix for applied migrations; do not delete/modify production history casually.
- Document gateway outage behaviour and manual operations fallback without recording false payments/redemptions.

5. India launch checks

The platform team must assess and obtain qualified advice for applicable marketplace, payment aggregation/settlement, consumer protection, taxation/invoicing, privacy/data protection, labour/partner and food-safety obligations before public launch. Verify provider food business licensing/registration and platform obligations applicable to the chosen operating model. Do not state that MESS is legally compliant merely because a field for a licence number exists.

Reference points to verify during launch:

- Supabase RLS and explicit grants: https://supabase.com/docs/guides/database/postgres/row-level-security
- Supabase Edge Functions/secrets: https://supabase.com/docs/guides/functions
- FSSAI business registration information: https://fssai.gov.in/business/registration
- FSSAI e-commerce food business operator guidance: https://www.fssai.gov.in/upload/advisories/2018/02/5a968f14cc994189.pdf
- Selected payment gateway's current official payment, marketplace, webhook, refund and settlement documentation.

These are links to official starting points, not legal advice. Requirements depend on the exact business and fund-flow model.

6. Release blockers

Do not enable production transactions until all are true:

- Payment gateway account and required marketplace/settlement features are approved/configured.
- Production secrets are stored server-side.
- Production data access and RLS tests pass.
- Refund/cancellation and customer terms are published.
- Provider terms, commission/promo acceptance and payout rules are recorded.
- Provider verification and applicable food-safety requirements are checked.
- Customer support and incident ownership are assigned.
- Payment/reconciliation, redemption and delivery failure tests pass.
- Backups, logging, monitoring and a rollback/forward-fix plan exist.
- A named human explicitly approves production launch.